Mentoring Session Booking System

A deep dive into multi-tenant architecture, concurrency-safe slot booking, and full-stack implementation

System Overview

Rails 8 API

Backend with pessimistic locking for concurrency-safe transactions

React 19

Modern frontend with real-time updates and responsive UX

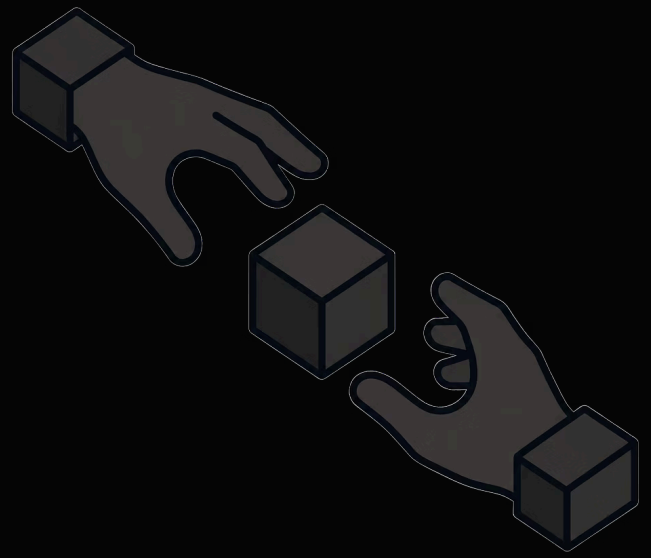
PostgreSQL 16 + Redis 7

Persistent storage paired with a high-performance caching layer

Sidekiq + Docker

Async job processing for notifications, one-command local dev

Built on deep scheduling domain experience from GetGist — scope cuts here were deliberate, not ignorant.



Core Problem: Concurrent Slot Booking

When multiple users attempt to book the same mentor slot simultaneously, race conditions emerge — and without a locking strategy, the result is double-bookings and data corruption.

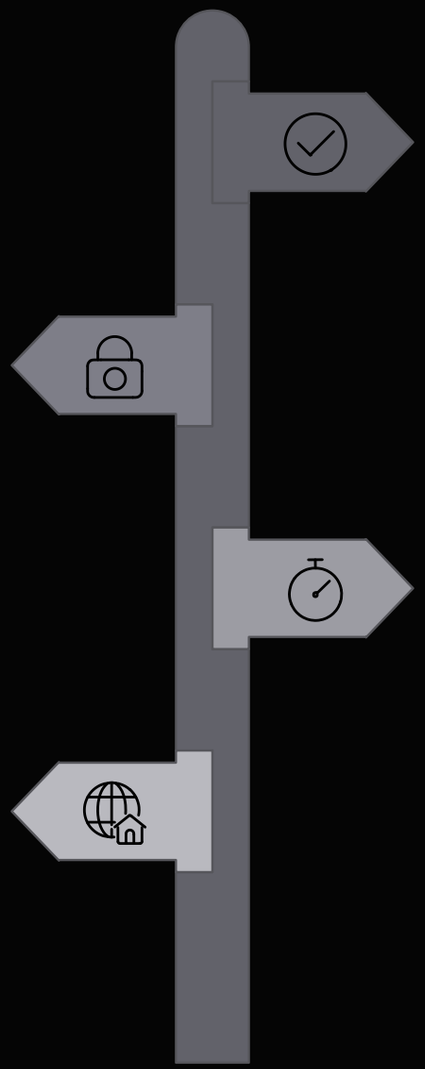
The Race Condition

Two requests read the same slot as *available* before either commits

The Solution

Pessimistic locking at the database level — acquire the lock, then act

Database Design: Two-State Slots



Available

Open for booking; no lock held.

Booked

Locked during transaction to prevent concurrent claims.

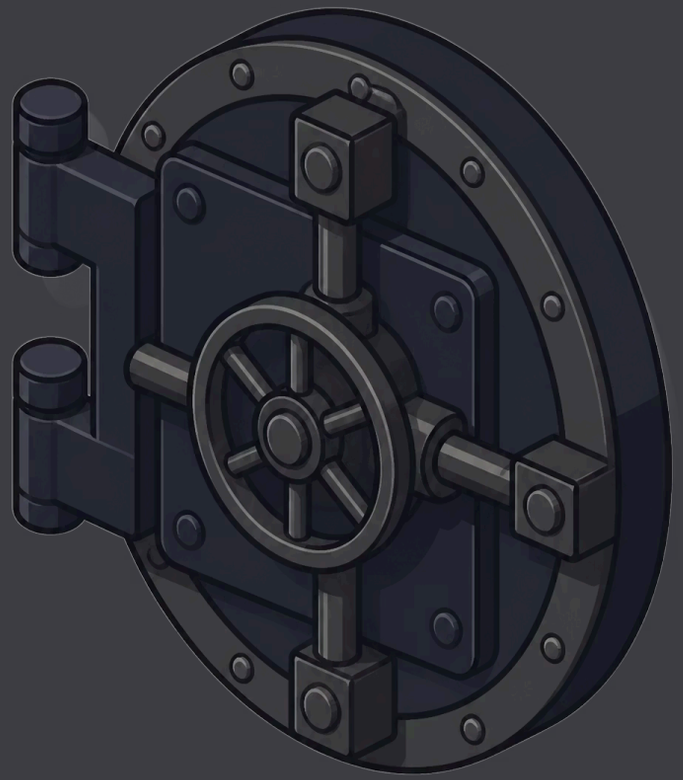
Buffer

buffer_minutes enforces spacing between sessions.

Tenant Isolation

acts_as_tenant ensures each slot belongs to one tenant.

Every slot belongs to exactly one tenant and carries a `buffer_minutes` field that enforces enforced spacing between adjacent sessions — preventing back-to-back overloads on a mentor's calendar.



Booking Flow: Pessimistic Lock in Action

01

Acquire the Lock

Use `lock!` to take an exclusive row-level lock.

02

Validate Buffer

Check nearby slots before moving forward.

03

Create Booking

Insert the booking atomically in the open transaction.

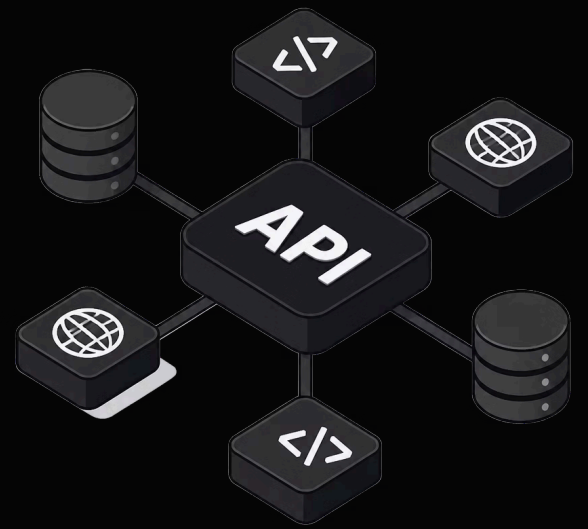
04

Release on Commit

Commit the transaction, then let queued requests continue.

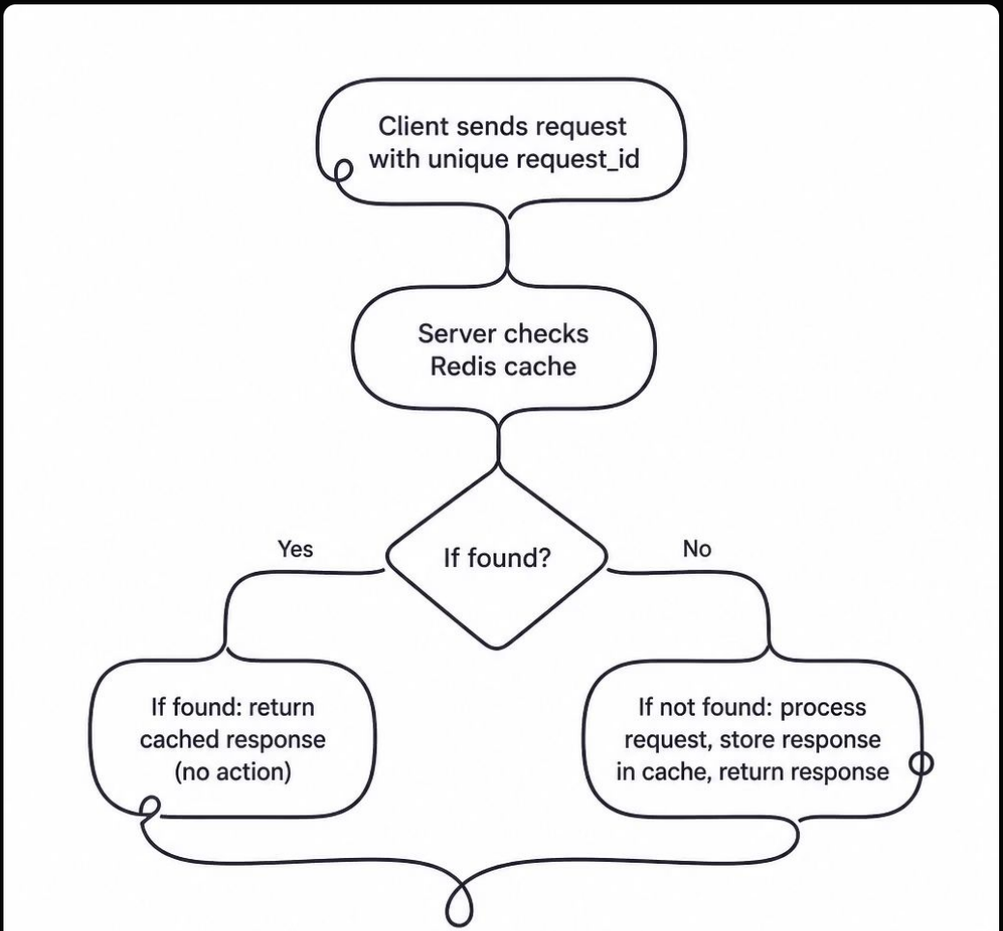
API Endpoints: RESTful Design

Method	Endpoint	Purpose
GET	/api/v1/mentors	Browse mentors by skills
GET	/api/v1/slots	List available slots
POST	/api/v1/bookings	Create booking (idempotent)
PATCH	/api/v1/bookings/:id/reschedule	Move to different slot
PATCH	/api/v1/bookings/:id/cancel	Cancel session



Idempotency: Handling Retries Safely

Network timeouts and client retries are inevitable at scale. Every mutating request carries a unique `request_id` header — the server caches the response and returns it on duplicates, guaranteeing no double-booking or double-charge.



Client Sends

Unique `request_id` on every POST / PATCH

Server Stores

Response cached against `request_id` in Redis

Duplicate Returns

Cached response — no side effects re-executed

Frontend: React 19 Architecture

Mentor Browse

Search by skills, filter by availability, view mentor profiles and calendars

Booking Flow

Select slot → confirm details → receive real-time confirmation feedback

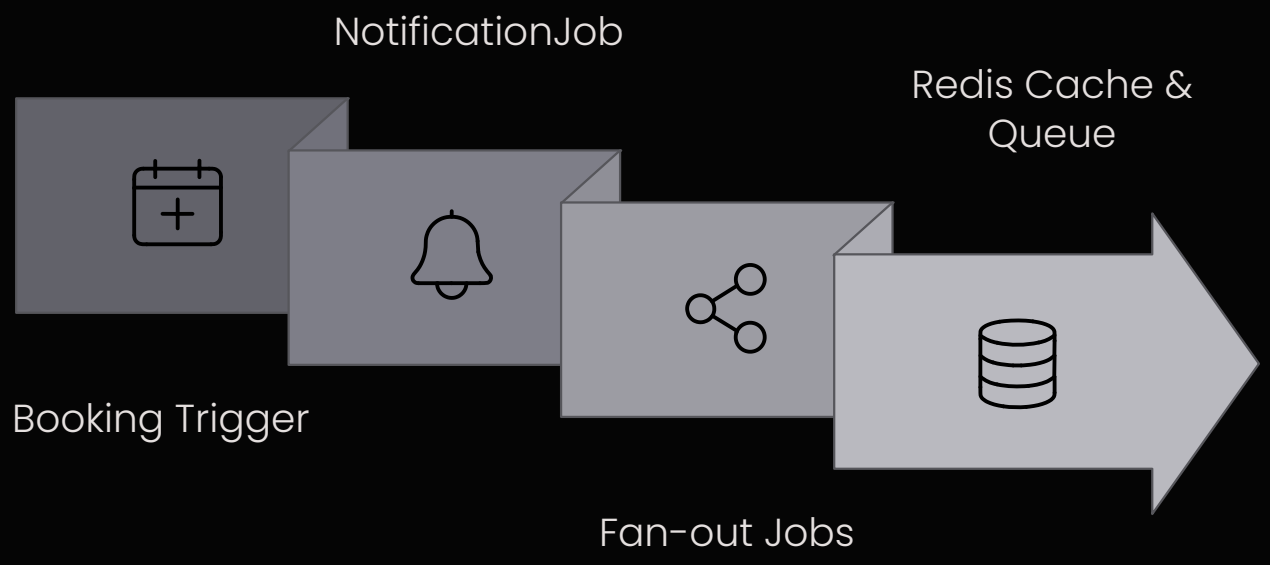
Session Management

View upcoming and past sessions, reschedule or cancel with one click

Notifications

In-app alerts for all booking lifecycle events — both parties notified

Async Processing: Sidekiq + Redis



Why Async?

Background jobs keep the booking API fast by moving slow work off the request path. The endpoint can return immediately while notifications, emails, and analytics continue in the background.

- **NotificationJob** — persists alerts to DB for both mentor and mentee
- **Email delivery** — queued via Sidekiq, retried on failure
- **Analytics + briefs** — generated asynchronously, never blocking UX
- **Redis dual role** — job queue backbone and slot availability cache

Multi-Tenancy: Isolation & Security

Every model is scoped through `acts_as_tenant` — tenant ID flows from the JWT token, enforced at the controller level before any query runs.

→ Automatic Scoping

`Booking.all` returns only the current tenant's records — always

→ JWT Enforcement

Tenant ID extracted from token, validated before any controller action

→ Zero Leakage

Cross-tenant data access is architecturally impossible at the query layer

Testing & Quality

204

RSpec Tests

Covering booking flows, concurrency scenarios, and API contracts

91.83%

Code Coverage

Critical paths fully tested — no happy-path-only shortcuts

0

RuboCop Offenses

Consistent style enforced across the entire codebase

Load tests validate the pessimistic locking strategy under real concurrent pressure — confirming zero double-bookings even at peak request volume.